Universidad de La Laguna

Grado en Ingeniería Informática

Computabilidad y Algoritmia

PRÁCTICA 6

Diseño e implementación de un simulador de DFAs

Factor de ponderación: 10

Semana del 19 al 23 de octubre de 2026

1. Objetivos

El objetivo de la práctica es consolidar conocimientos sobre Autómatas Finitos (FA, del inglés *Finite Automata*) al mismo tiempo que se continúan desarrollando capacidades para diseñar y desarrollar programas orientados a objetos en C++. Mientras que en la anterior práctica se utilizó una herramienta externa para simular autómatas finitos deterministas (DFAs) y automátas finitos no deterministas (NFAs), en esta práctica se desarrollará un programa que permita simular cualquier DFA especificado mediante un fichero de entrada. El programa leerá desde un fichero las especificaciones de un DFA y, a continuación, simulará el comportamiento del autómata para las cadenas que se suministren como entrada, también a través de un fichero.

Al igual que en prácticas anteriores, se propone que el alumnado utilice este ejercicio para poner en práctica aspectos generales relacionados con el desarrollo de programas en C++. Algunos de estos principios que enfatizaremos en esta ocasión serán los siguientes:

- **Programación orientada a objetos**: es fundamental identificar y definir clases y objetos que permitan modelar adecuadamente el escenario de trabajo que se plantea.
- **Diseño modular**: el programa debiera escribirse de modo que las diferentes funcionalidades que se precisen sean encapsuladas en métodos concretos cuya extensión textual se mantenga acotada.
- **Pautas de estilo**: es imprescindible ceñirse al formato para la escritura de pro-

gramas en C++ que se propone en esta asignatura [7]. Algunos de los principales criterios de estilo ya se han descrito reiteradamente en prácticas anteriores.

- **Documentación**: se requiere que los comentarios del código fuente sigan el formato especificado por Doxygen [2]. Además, se propone seguir esta referencia [3] para mejorar la calidad de la documentación de su código fuente.
- Compilación de programas utilizando `make` [5, 6].

2. Autómatas Finitos Deterministas

Un Autómata Finito Determinista (DFA) está especificado por un conjunto de estados y un conjunto de transiciones entre estados que se producen al procesar símbolos de entrada pertenecientes a un alfabeto Σ. Para cada símbolo de entrada hay exactamente una transición para cada estado. Hay un estado especial, denominado estado inicial o de arranque que es el estado en el que el autómata se encuentra inicialmente. Por otra parte, algunos estados se denominan finales o de aceptación. Así pues, un Autómata Finito Determinista se caracteriza formalmente por una quíntupla $(\Sigma, Q, q_0, F, \delta)$ donde cada uno de estos elementos tiene el siguiente significado:

- Σ es el alfabeto de entrada del autómata. Se trata del conjunto de símbolos que el autómata acepta como entradas.
- $Q \neq \emptyset$ es el conjunto finito de estados del autómata, que siempre se encontrará en uno de los estados de Q.
- $q_0 \in Q$ es el estado inicial o de arranque del autómata. Se trata de un estado distinguido en el que se encuentra el DFA al comienzo de la ejecución.
- $F \subseteq Q$ es el conjunto de estados finales o de aceptación del autómata. Al final de una ejecución, si el estado en que se encuentra el autómata es un estado final, diremos que el autómata ha aceptado la cadena de símbolos de entrada.
- δ es la función de transición. $\delta : Q \times \Sigma \to Q$ que determina el único estado siguiente para un par (q_i, σ) correspondiente al estado actual y la entrada.

El siguiente es un primer ejemplo de DFA definiendo los cinco elementos que lo componen:

- $\Sigma = \{a, b\}$
- $Q = \{q_0, q_1, q_2\}$
- q_0
- $F = \{q_0\}$
- La función de transición, δ está definida en la Tabla 1, en la que la primera columna contiene los estados posibles del autómata, y la primera fila todos los símbolos del alfabeto. Dado un estado actual q y un símbolo de entrada $\sigma \in \Sigma$, la tabla define el

estado al que transita el autómata cuando estando en el estado q, recibe la entrada σ: $\delta(q, \sigma) \in Q$

δ	a	b
q_0	q_1	q_2
q_1	q_2	q_0
q_2	q_2	q_2

Tabla 1: La función de transición δ

La característica definitoria de un DFA es que δ es una función, por lo que debe estar definida para todos los pares del producto cartesiano $Q \times \Sigma$. Por ello, sea cual fuere el símbolo actual de la entrada y el estado en que se encuentre el autómata, siempre hay un estado siguiente y además este estado se determina de forma unívoca. Dicho de otro modo, la función de transición siempre determina unívocamente el estado al que ha de transitar el autómata dados el estado en que se encuentra y el símbolo que se tiene en la entrada.

Con un DFA siempre se puede asociar un grafo dirigido que se denomina diagrama de transiciones. Su construcción es como sigue: los vértices del grafo se corresponden con los estados del DFA. Si hay una transición desde el estado q hacia el estado p con la entrada i, entonces deberá haber un arco desde el nodo q hacia el nodo p etiquetado i. Los estados de aceptación se representan mediante un círculo de doble trazo, y el estado de arranque se indica mediante una flecha dirigida hacia el correspondiente vértice.

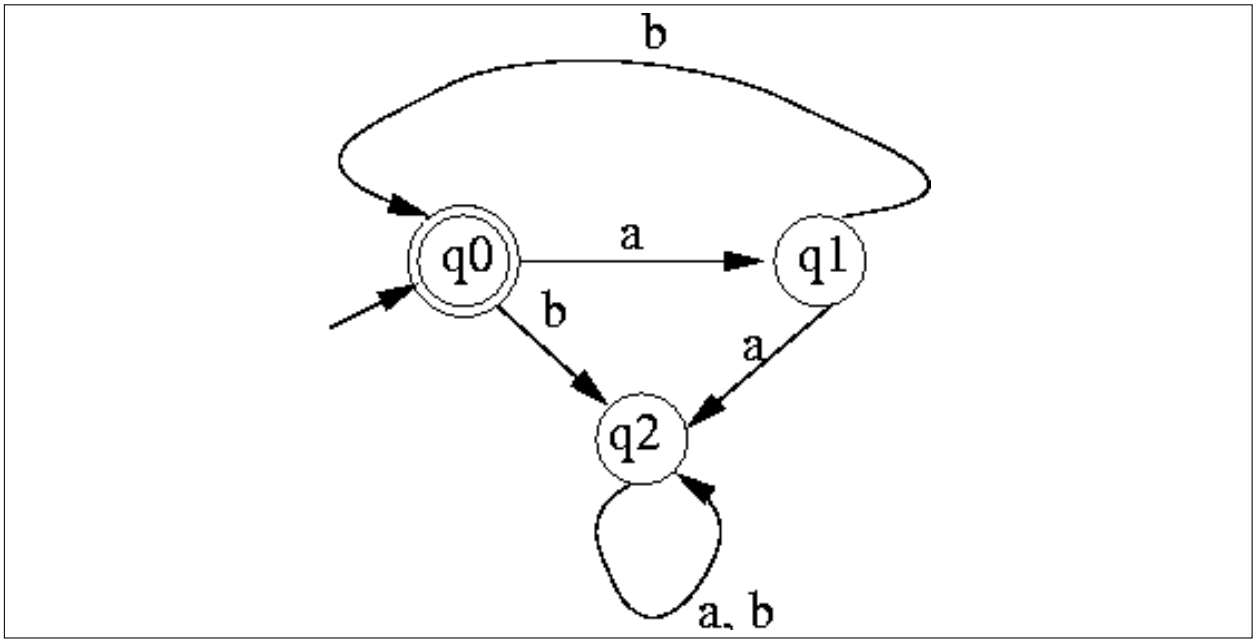


Figura 1: Diagrama de transiciones del DFA especificado en la Tabla 1

El diagrama de transiciones correspondiente al DFA de la Tabla 1 es el que se muestra en la figura 1. En esa figura se observa que el estado inicial, q_0 es el único estado de

aceptación. El estado inicial está indicado por la flecha que entra hacia él, sin partir de otro estado.

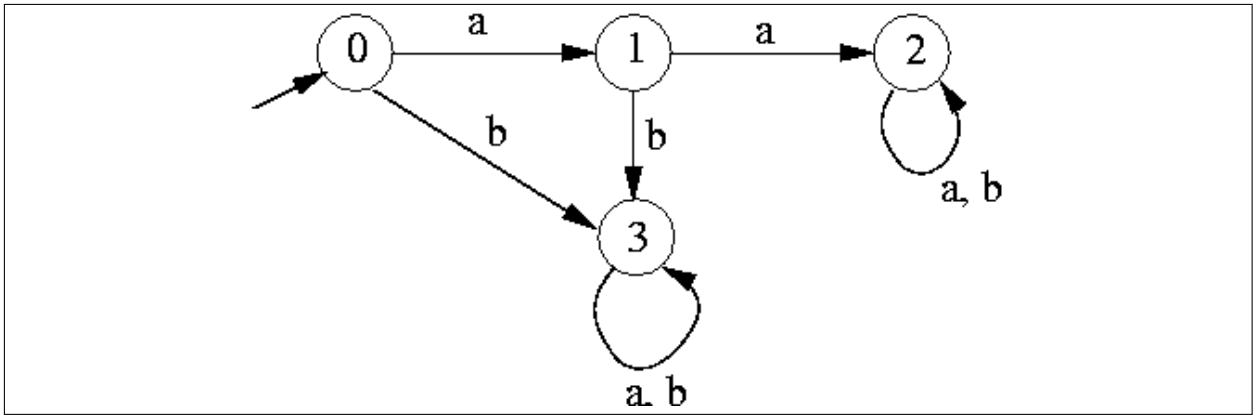


Figura 2: El estado 3 es un estado de absorción

Se dice que un estado de un DFA es un estado de absorción si el estado no es de aceptaación y todas las transiciones de ese estado para cualquier símbolo del alfabeto llevan a ese mismo estado. Por ejemplo, el estado 3 del DFA de la Figura 2 es un estado de absorción. Por convenio, a la hora de representar un autómata se supone que las transiciones de un determinado estado que no aparecen dibujadas con arcos en el diagrama de transiciones, conducen a un estado de muerte, que no se dibuja. Así el DFA de la Figura 2 se puede representar como aparece en la Figura 3, en la que el estado 3 no aparece dibujado explícitamente.

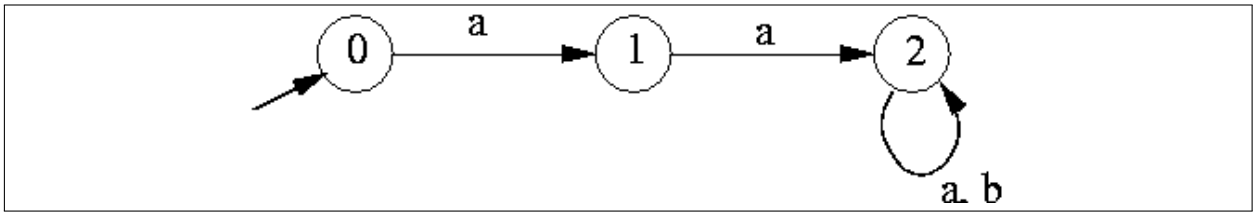


Figura 3: El autómata de la Figura 2 sin el estado de muerte

3. Lenguaje aceptado por un DFA

Para describir formalmente el comportamiento del autómata ante una cadena $w \in \Sigma^*$ debemos ampliar la función de transición $\delta : Q \times \Sigma \to Q$ para aplicarla sobre un estado y una cadena de símbolos en lugar de un estado y un símbolo del alfabeto. Se define entonces $\hat{\delta} : Q \times \Sigma^* \to Q$

La idea es que $\hat{\delta}(q, w)$ será el estado que alcance el DFA después de leer la cadena w arrancando en el estado q. Dicho de otro modo, $\hat{\delta}(q, w)$ es el único estado p tal que hay

un camino en el diagrama de transiciones del autómata desde q hasta p etiquetado con los símbolos de w. Formalmente se define:

1. $\hat{\delta}(q, e) = q$
2. Para todas las cadenas $w \in \Sigma^*$ y símbolos $a \in \Sigma$: $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$

El primer punto establece que el autómata no puede cambiar de estado sin leer ningún símbolo, y el segundo indica cómo decidir el estado en que se encuentra el autómata después de leer una cadena no vacía wa.

Dado que $\hat{\delta}(q, a) = \delta(\hat{\delta}(q, e), a) = \delta(q, a)$ las funciones δ y $\hat{\delta}$ ofrecen el mismo resultado sobre argumentos válidos para ambas (δ toma como entrada un par estado, símbolo del alfabeto mientras que $\hat{\delta}$ toma un estado y una cadena de símbolos). Por eso se utiliza indistintamente δ para las funciones de transición de un autómata ya sea para una cadena o un único símbolo.

Se dice que una cadena $w \in \Sigma^*$ es aceptada por un autómata $M \equiv (\Sigma, Q, q_0, F, \delta)$ si $\delta(s, w) = p$ para algún estado $p \in F$. El lenguaje aceptado por M, que se designa como $L(M)$ es el conjunto de cadenas $\{w | \delta(q_0, w) \in F\}$. Se dice que un lenguaje es regular si es el conjunto de cadenas aceptado por un DFA. Es importante aclarar que L(M) es el conjunto de todas las cadenas que el autómata acepta (no un conjunto de cadenas que acepta).

4. Ejercicio práctico: simulación de un DFA

Esta práctica consistirá en la realización de un programa escrito en C++ que lea desde un fichero las especificaciones de un DFA y, a continuación, simule el comportamiento del autómata para una serie de cadenas que se suministren como entrada.

El programa debe ejecutarse como:

```
$ ./p06_DFA_simulator input.dfa input.txt
```

Donde los dos parámetros pasados en la línea de comandos corresponden en este orden con:

- Un fichero de texto con extensión `.dfa` en el que figura la especificación de un DFA.
- Un fichero de texto con extensión `.txt` en el que figura una serie de cadenas (una cadena por línea) sobre el alfabeto del autómata especificado en el primero de los ficheros.

Tal y como se ha venido recomendando en las prácticas de la asignatura, el comportamiento del programa al ejecutarse en línea de comandos debiera ser similar al de los comandos de Unix. Así por ejemplo, si se ejecuta el programa sin parámetros, se debería obtener información sobre el uso correcto del simulador:

```
$ ./p06_DFA_simulator
Modo de empleo: ./p06_DFA_simulator input.dfa input.txt
Pruebe 'p06_DFA_simulator --help' para más información.
```

La opción `--help` en línea de comandos ha de producir que se imprima en pantalla un breve texto explicativo del funcionamiento del programa. Una información que puede ser de especial ayuda para los usuarios del simulador sería precisamente el formato del fichero que contendrá la especificación del autómata.

4.1. Estructura de los ficheros de definición de DFAs

```
//
// DFA que reconoce el lenguaje representado por la expresi n (a|b)*abb
//
ab
5
0
0 0 a 1 b 2
1 0 a 1 b 3
2 0 a 1 b 2
3 0 a 1 b 4
4 1 a 1 b 2
```

Listado 1: Fichero de definición del DFA de la figura 4

Estos ficheros tendrán la extensión `.dfa` y el siguiente formato:

- Línea 1: Símbolos del alfabeto Σ del autómata. Serán caracteres alfanuméricos (letras o dígitos) escritos en una única línea sin espacios de separación entre ellos. Cada símbolo se corresponderá con un único caracter alfanumérico.
- Línea 2: El número total de estados del DFA.
- Línea 3: Estado de arranque del DFA.

A continuación habrá una línea para cada estado, conteniendo los siguientes números, separados entre sí por espacios en blanco:

- Número identificador del estado.
- Un 1 si se trata de un estado de aceptación o un 0 en caso contrario.

A continuación, en la misma línea, para cada una de las transiciones y separados por espacios:

- Símbolo de entrada necesario para que se produzca la transición.
- Estado destino de la transición.

En los ficheros de definición de DFAs, las líneas que contengan el prefijo `//` deberán considerarse líneas de comentarios, y por tanto serán ignoradas. Se utilizan solamente

para hacer anotaciones en el fichero, que no repercuten en la estructura del DFA.

El listado 1 presenta el fichero de definición del DFA de la Figura 4.

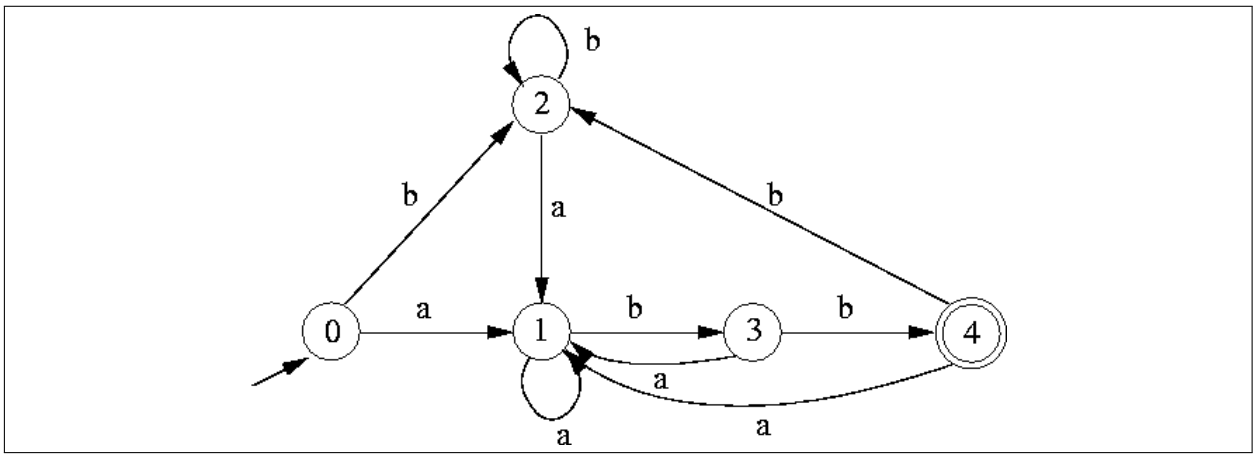


Figura 4: DFA que reconoce las cadenas del lenguaje descrito por `(a|b)*abb`

4.2. Ficheros de entradas

Estos ficheros tendrán la extensión `.txt` y las cadenas de entrada al autómata figuran en el fichero sin separación entre los símbolos de la misma. Así para el autómata de la Figura 4 podríamos tener las entradas que se muestran en el Listado 2.

```
ababbbbabb
abbbbababa
```

Listado 2: Fichero con entradas para autómatas

Para representar la cadena vacía se usará el símbolo `&`.

```
Est. Actual Entrada   Nuevo Est.
    0       a         1
    1       b         3
    3       a         1
    1       b         3
    3       b         4
    4       b         2
    2       b         2
    2       a         1
    1       b         3
    3       b         4

Cadena de entrada ACEPTADA.
```

Listado 3: Resultado de la ejecución del programa

4.3. Trazas

A medida que el autómata lea entradas deberá imprimirse una lista que contenga el número identificador del estado actual, la entrada leída y el nuevo número identificador del estado al que se transita, finalizando todo ello con un mensaje que indique si se ha aceptado la cadena de entrada o no. Así para la primera de las entradas del Listado 2 obtendríamos los resultados que se presentan en el Listado 3.

5. Consideraciones de implementación

El programa principal deberá crear un DFA a partir de la especificación dada en el fichero `.dfa` pasado por línea de comandos. Habrá que notificar al usuario si se produce algún error en la creación del autómata. Esto es, el programa deberá detectar y notificar adecuadamente al usuario en caso de que se detecte algún error o inconsistencia en la definición del autómata. Algunos ejemplos de condiciones que debe cumplir todo fichero de especificación son:

- Existe uno y sólo un estado inicial para el autómata.
- No existen transiciones que tengan lugar con símbolos no pertenecientes al alfabeto del DFA.
- Hay una línea en el fichero por cada uno de los estados del autómata.
- Los estados a los que se transita son estados que forman parte del propio autómata.

Si el autómata ha sido creado correctamente, entonces se procederá a leer una a una las cadenas contenidas en el fichero `.txt` de entrada. Para cada cadena en el fichero de entrada se deberá simular el comportamiento del autómata para determinar si dicha cadena es aceptada o rechazada por el autómata.

La idea central de la Programación Orientada a Objetos, OOP (*Object Oriented Programming*) es dividir los programas en piezas más pequeñas y hacer que cada pieza sea responsable de gestionar su propio estado. De este modo, el conocimiento sobre la forma en que funciona una pieza del programa puede mantenerse local a esa pieza. Alguien que trabaje en el resto del programa no tiene que recordar o incluso ser consciente de ese conocimiento. Siempre que estos detalles locales cambien, sólo el código directamente relacionado con esos detalles precisa ser actualizado.

Las diferentes piezas de un programa de este tipo interactúan entre sí a través de lo que se llama interfaces: conjuntos limitados de funciones que proporcionan una funcionalidad útil a un nivel más abstracto, ocultando su implementación precisa. Tales piezas que constituyen los programas se modelan usando objetos. Su interfaz consiste en un conjunto específico de métodos. Separar la interfaz de la implementación es una buena práctica que se denomina encapsulamiento.

C++ es un lenguaje orientado a objetos. Si se hace programación orientada a objetos, los programas forzosamente han de contemplar objetos, y por tanto clases. Cualquier programa que se haga ha de modelar el problema que se aborda mediante la definición de clases y los correspondientes objetos. Los objetos pueden componerse: un objeto "coche" está constituido por objetos "ruedas", "carrocería" o "motor". La herencia es otro mecanismo que permite hacer que las clases (objetos) herederas de una determinada clase posean todas las funcionalidades (métodos) y datos (atributos) de la clase de la que descienden. De forma inicial es más simple la composición de objetos que la herencia, pero ambos conceptos son útiles a la hora de desarrollar aplicaciones complejas. Cuanto más compleja es la aplicación que se desarrolla mayor es el número de clases involucradas. En los programas que desarrollará en esta asignatura será frecuente la necesidad de componer objetos, mientras que la herencia tal vez tenga menos oportunidades de ser necesaria.

Tenga en cuenta las siguientes consideraciones:

- No traslade a su programa la notación que se utiliza en este documento, ni en la teoría de Autómatas Finitos. Por ejemplo, el conjunto de estados de su autómata no debiera almacenarse en una variable cuyo identificador sea `Q`. Al menos por dos razones: porque no sigue lo especificado en la *guía de estilo* [7] respecto a la elección de identificadores y más importante aún, porque no es significativo. No utilice identificadores de un único carácter, salvo para situaciones muy concretas.
- Favorezca el uso de las clases de la STL, particularmente `std::set`, `std::map` y `std::multimap`, frente al uso de estructuras dinámicas de memoria gestionadas a través de punteros.
- Construya su programa de forma incremental y depure las diferentes funcionalidades que vaya introduciendo.
- En el programa parece ineludible la necesidad de desarrollar una clase `Automata`. Estudie las componentes que definen a un autómata y vea cómo trasladar esas componentes a su clase.
- Valore análogamente qué otras clases se identifican en el marco del problema que se considera en este ejercicio. Estudie esta referencia [4] para practicar la identificación de clases y objetos en su programa.

6. Criterios de evaluación

Se señalan a continuación los aspectos más relevantes (la lista no es exhaustiva) que se tendrán en cuenta a la hora de evaluar esta práctica:

- Se valorará que el alumnado haya realizado, con anterioridad a la sesión de prácticas, y de forma efectiva, todas las tareas propuestas en este guión. Esto implicará que el programa compile y ejecute correctamente. Además, el comportamiento del programa deberá ajustarse a lo solicitado en este documento.
- También se valorará que, con anterioridad a la sesión de prácticas, el alumnado haya revisado los documentos que se enlazan desde este guión.
- Paradigma de programación orientada a objetos: el programa ha de ser fiel al paradigma de programación orientada a objetos. Se valorará que el alumnado haya identificado clases y objetos que permitan modelar adecuadamente el escenario de trabajo que se plantea.
- Paradigma de modularidad: se valorará que el programa se haya escrito de modo que las diferentes funcionalidades que se precisen hayan sido encapsuladas en métodos concretos cuya extensión textual se mantuviera acotada.
- Documentación: se requiere que todos los atributos de las clases definidas en el proyecto tengan un comentario descriptivo de la finalidad del atributo en cuestión. Además, se requiere que los comentarios del código fuente sigan el formato especificado por Doxygen [2].
- Se valorará que el código desarrollado siga el formato propuesto en esta asignatura para la escritura de programas en C++.
- Capacidad de la programadora de introducir cambios en el programa desarrollado.

Si el alumnado tiene dudas respecto a cualquiera de estos aspectos, debiera acudir al foro de discusiones de la asignatura para plantearlas allí. Se espera que, a través de ese foro, el alumnado intercambie experiencias y conocimientos, ayudándose mutuamente a resolver dichas dudas. También el profesorado de la asignatura intervendrá en las discusiones que pudieran suscitarse, si fuera necesario.

Referencias

[1] Transparencias del Tema 2 de la asignatura: Autómatas finitos y lenguajes regulares. (Disponibles en el aula virtual de la asignatura) https://drive.google.com/file/d/1Wv47hxpcC0qhGDIDVCXlgPTqzVreULnh/view?usp=sharing

[2] Doxygen http://www.doxygen.nl/index.html

[3] Diez consejos para mejorar tus comentarios de código fuente https://www.genbeta.com/desarrollo/diez-consejos-para-mejorar-tus-comentarios-de-codigo-fuente

[4] Cómo identificar objetos en el modelado de un programa https://www.studocu.com/es-ar/document/universidad-de-buenos-aires/diseno-i/como-identificar-objetos-en-el-modelado-de-un-programa/28183691

[5] Makefile Tutorial: http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor

[6] C++ Makefile Tutorial: https://makefiletutorial.com

[7] Google C++ Style Guide, https://google.github.io/styleguide/cppguide.html